Ускорение инференса UNet модели

Бенчмарк оптимизаций: от PyTorch до TensorRT INT8 + Sparse 2:4

Крамаренко Георгий
Яскевич Михаил
Липилин Матвей

Задача и окружение

Оптимизировать инференс модели UNet:

- Компиляция
- Квантизация
- Спарсификация

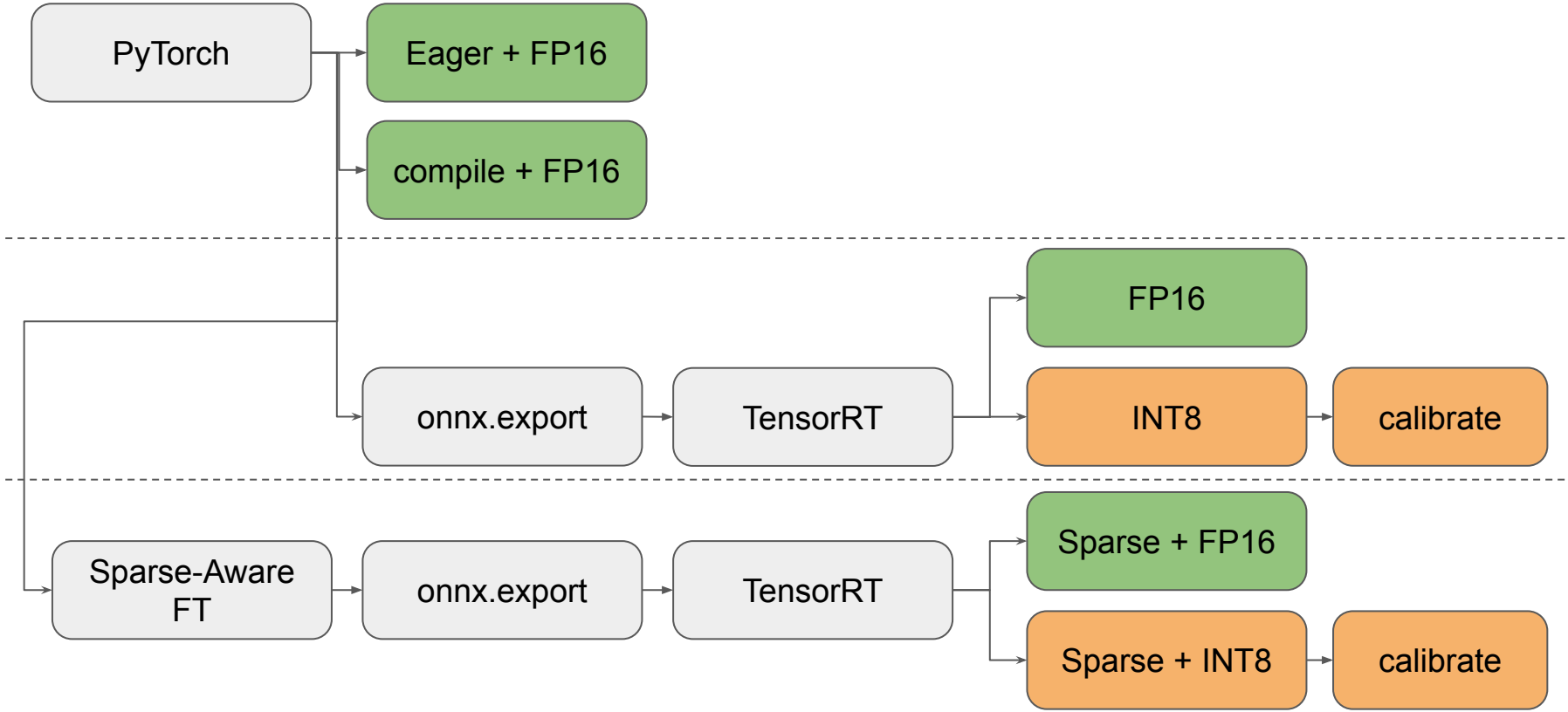
Датасет:

- Oxford-IIIT Pet Dataset - 37 пород, ~7400 изображений с масками (3 класса: питомец / фон / граница)

Окружение:

- GPU: NVIDIA A100-PCIE-40GB / Version: 580.126
- CUDA: 13.2 / TensorRT: 10.16.1.11
- PyTorch: 2.11.0 (16-mixed)

Пайплайн оптимизации



Квантизация

- В **FP16** PyTorch / TensorRT - легко
- В **INT8** TensorRT - требует калибровки для уменьшения потери качества: генерируем **калибровочный кеш** и используем его для последующей квантизации *trtexec*

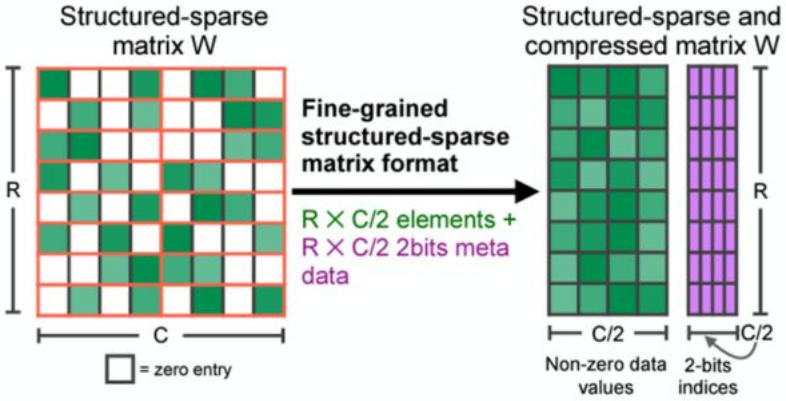
```
# 1. Калибровка: 16 батчей × 32 картинки из test_loader → calib.cache
python3 calibrate.py \
    --onnx experiments/trt_fp16/unet.onnx \
    --cache experiments/trt_fp16/calib_dense.cache

# 2. Build int8 engine с этим cache
trtexec --onnx=experiments/trt_fp16/unet.onnx \
    --saveEngine=experiments/trt_fp16/unet_int8.engine \
    --fp16 --int8 --calib=experiments/trt_fp16/calib_dense.cache \
    --minShapes=input:1x3x256x256 --optShapes=input:32x3x256x256 --maxShapes=input:32x3x256x256 \
    --profilingVerbosity=detailed --skipInference
```

Спарсификация

Используем Structured Sparsity 2:4

- Sparse-Aware FT (3 эпохи) - дообучение с масками спарсификации
- На выходе - чекпоинт, в котором ровно **49.98%** весов Conv2d/Linear = 0.0, выровненных по **2:4**
- Перевод в TensorRT с флагом `***--sparsity=enable***`



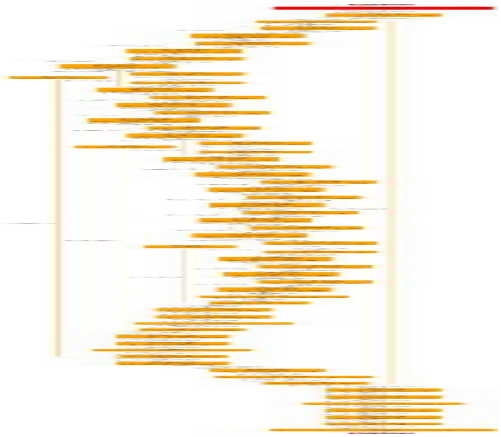
Результаты

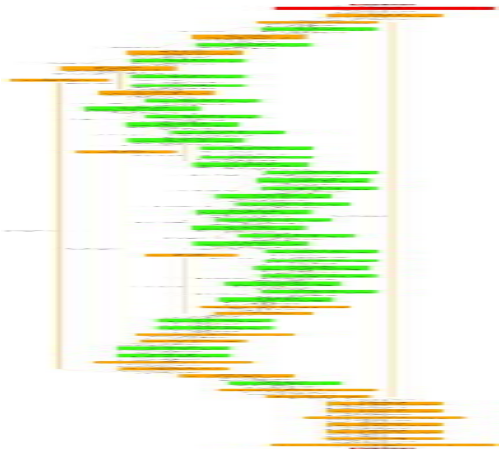
Метод	val_iou	Latency, ms	Throughput, img/s	Peak GPU, MB	Speedup
fp16 baseline	0.8732	11.25	2844	1623	1.00×
torch.compile	0.8732	9.87	3241	1506	1.14×
trt fp16	0.8732	5.09	6283	108	2.21×
trt int8	0.8730	2.96	10799	132	3.80×
trt fp16 sparse 2:4	0.8683	4.51	7103	156	2.50×
trt int8 sparse 2:4	**0.8680**	**2.69**	**11888**	**180**	**4.18×**

Инсайты по спарсификации

Мы визуализировали графы и инференса TensorRT engine для **dense** и **sparse** режимов через **trex:**
Оранжевый - dense-ядра, Зеленый - sparse-ядра

Наблюдение: на некоторых слоях не выгодно использовать sparse-ядра => эти слои можно было не спарсифицировать





Выводы

NVIDIA TensorRT + INT8 + A100 Spare Tensor Cores по сравнению с обычным PyTorch FP16 позволяет:

- ускорить инференс в **x4 раза**
- сократить пиковое потребление VRAM в **x8 раз**
- все это при погрешности *val_iou* в **5e-3**

Спасибо за внимание